

AI Neural Network Simulator

Heart Disease Prediction using a Neural Network from Scratch

A Semester Project Report

Course: CSE142 Object Oriented Programming Techniques
Semester: Spring 2026
Instructor: Sir Behraj Khan
Due Date: 10 May 2026
Author(s): Laiba Irfan ERP: 31736
 Labina Abbasi ERP: 32522
 Muhammad Haris Shaikh ERP: 33480

IBA KARACHI

Institute of Business
Administration

Department of Computer Science
Institute of Business Administration, Karachi

May 10, 2026

Abstract

This report describes the **AI Neural Network Simulator**, a heart-disease prediction program built from scratch in C++17. It trains a feedforward neural network ($16 \rightarrow 16 \rightarrow 8 \rightarrow 1$) using backpropagation and gradient descent on 950 patient records, each with 16 clinical features. The code is structured around OOP: an abstract `Layer` class is extended by `DenseLayer` and `SigmoidLayer`; a `Matrix` class handles all math with overloaded `+`, `-`, `*`, `==`, and `<<` operators; and `NeuralNetwork` manages the layers using `std::unique_ptr`. Other classes handle CSV loading, Xavier weight setup, top-3 epoch tracking with `std::priority_queue`, and saving/loading the model. A live SFML 3 window (1200×800) shows the network activating in real time, with a dashboard displaying predictions, loss, and a 60-epoch loss graph. All eleven classes together cover encapsulation, inheritance, polymorphism, and operator overloading.

Keywords: Object-Oriented Programming, C++17, Neural Network, Heart Disease Prediction, Backpropagation, SFML 3, Polymorphism, Operator Overloading, Smart Pointers

Contents

Abstract	2
1 Introduction	4
1.1 Background	4
1.2 Problem Statement	4
1.3 Objectives	4
1.4 Scope and Limitations	4
2 OOP Design	4
2.1 Class Hierarchy	4
2.2 Class Descriptions	6
2.3 OOP Concepts Applied	6
3 Implementation	7
3.1 Development Environment	7
3.2 Abstract Layer Hierarchy	7
3.3 Matrix Operator Overloading	8
3.4 Forward and Backward Propagation	9
3.5 Xavier Weight Initialisation	10
3.6 Top- <i>K</i> Epoch Tracking with <code>std::priority_queue</code>	10
3.7 Network Construction and Entry Point	11
4 Testing	11
4.1 Test Cases	12
4.2 Edge Cases	12
5 Results and Discussion	13
5.1 Results	13
5.2 Analysis	13
6 Challenges and Solutions	13
6.1 Challenge 1: Storing Mixed Layer Types	13
6.2 Challenge 2: Animation Speed Tied to Frame Rate	14
7 Conclusion	14
References	15
A Complete Source Code	16
A.1 main.cpp	16
A.2 core_engine.h	16
A.3 core_engine.cpp	18
A.4 pipeline.h	20
A.5 interface.h	22
A.6 pipeline.cpp	23
A.7 interface.cpp	29
B Installation and Usage	38
B.1 Prerequisites	38
B.2 Build Instructions	38

1 Introduction

1.1 Background

This project builds such a classifier from scratch in C++17 without using any external ML library. Every matrix operation, gradient step, and training loop is written by hand, making it a hands-on application of the OOP concepts covered in CSE142 Object Oriented Programming Techniques.

1.2 Problem Statement

Given a patient record with 16 clinical features—age, sex, chest pain type, resting blood pressure, cholesterol, fasting blood sugar, resting ECG, max heart rate, exercise-induced angina, ST depression, ST slope, number of major vessels, thalassemia type, BMI, smoking history, and diabetes status—the network must output a binary label: 0 = healthy, 1 = disease.

- **Input:** A 1×16 row of feature values pre-scaled to $[0, 1]$.
- **Output:** A value in $(0, 1)$; anything above 0.5 is classified as disease.
- **Constraints:** No external ML or math libraries; C++ standard library only.

1.3 Objectives

1. Build a layer hierarchy using abstract base classes and polymorphism.
2. Write a `Matrix` class with operator overloading for clean math code.
3. Build a training pipeline with Xavier initialization, SGD, and MSE loss.
4. Show the network running live in an SFML 3 window.
5. Track the top-3 best training epochs using a `std::priority_queue`.
6. Save and load trained weights so progress is not lost on exit.

1.4 Scope and Limitations

Included: Feedforward inference, backpropagation, sigmoid activation, MSE loss, fixed-rate SGD, Xavier initialization, live SFML 3 visualization, model save/load.

Not included: GPU support, mini-batch training, convolutional/recurrent layers, dropout, learning-rate schedules, multi-class output.

Assumptions: Feature values are already scaled to $[0, 1]$; training uses one CPU thread; the CSV has a header row (`has_header = true`).

2 OOP Design

2.1 Class Hierarchy

The project has eleven classes. The layer classes use inheritance, while the training and UI classes are connected through composition. Figure 1 shows how they relate.

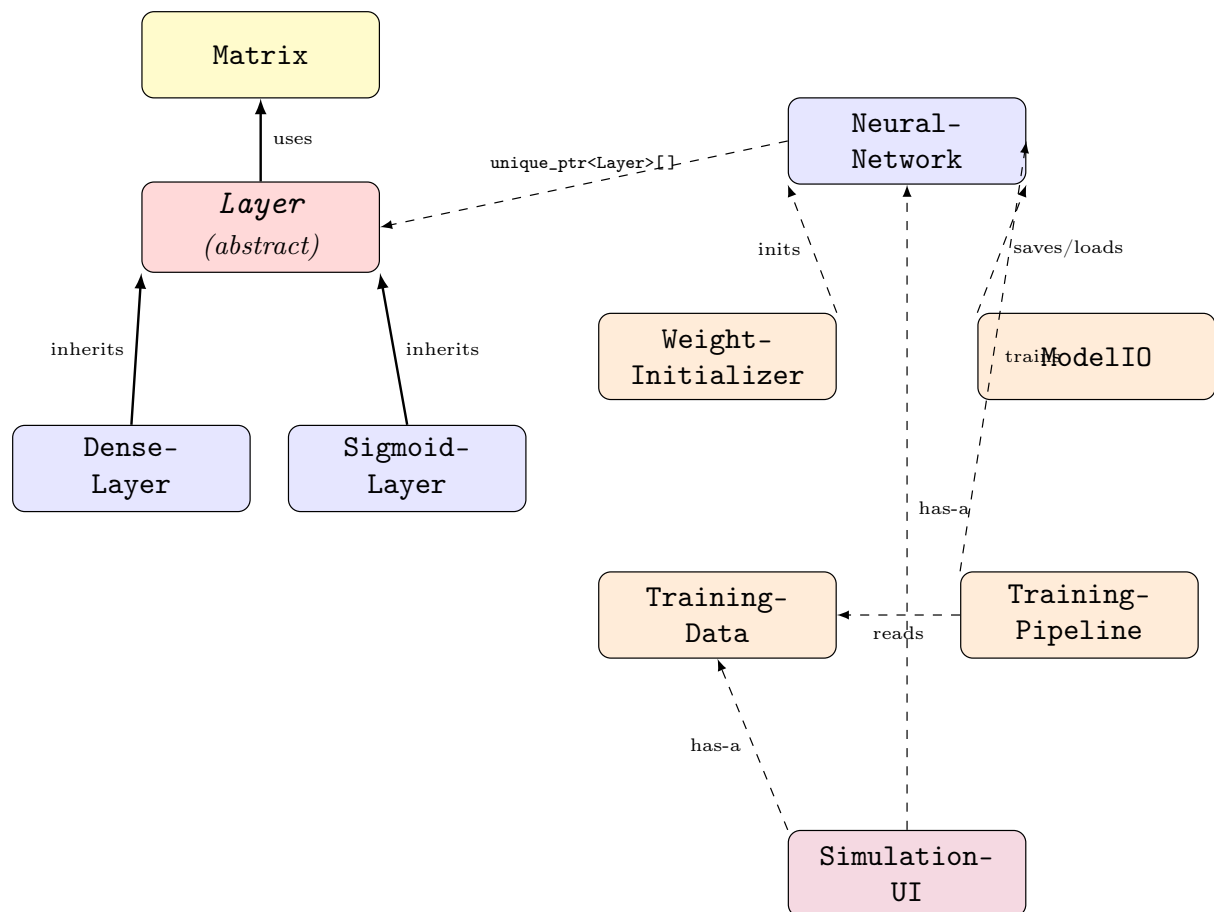


Figure 1: Class hierarchy and composition relationships

2.2 Class Descriptions

Table 1: Summary of all eleven project classes

Class	Responsibility	Key Members / Methods
Matrix	2-D numerical array; foundation of all data and gradient flow	rows, cols, data; operator+, -, *, ==, <; transpose(), randomize(), multiply()
Layer	Abstract interface every network layer must implement	last_input; pure-virtual forward(), backward(), get_type()
DenseLayer	Fully connected linear transform: $y = xW + b$	weights ($n_{in} \times n_{out}$), bias ($1 \times n_{out}$); set_weights()
SigmoidLayer	Element-wise sigmoid activation; no learnable parameters	Overrides forward() and backward() only
NeuralNetwork	Owns and sequences the layer stack; runs forward/backward passes	Private layers (vector<unique_ptr<Layer>); add_layer(), predict(), train()
TrainingSample	One dataset row: an input-target matrix pair	input (1×16), target (1×1)
TrainingData	Loads and stores the full dataset from a CSV file	Private samples; load_csv(), get_samples(), size()
TrainingPipeline	Runs training epochs; tracks per-epoch loss and top-K epochs	Static train() returns vector<double> loss history
WeightInitializer	Applies Xavier or uniform initialisation to all dense layers	Static xavier(), xavier_network(), uniform()
ModelIO	Saves/loads dense-layer weights to a versioned text file	Static save_model(), load_model()
SimulationUI	SFML 1200×800 real-time animated visualizer and dashboard	lossHistory (deque), bestEpochs (priority queue), run()

2.3 OOP Concepts Applied

- **Encapsulation:** NeuralNetwork keeps its layers vector private and only exposes predict(), train(), add_layer(), and get_layers(). TrainingData hides the samples and column counts so nothing outside can accidentally change the dataset.
- **Inheritance:** DenseLayer and SigmoidLayer both inherit from the abstract Layer class, which declares three pure-virtual functions—forward(), backward(), and get_type()—that every subclass must implement.

- **Polymorphism:** `NeuralNetwork` holds a `vector<unique_ptr<Layer>>` and calls `layer->forward()` and `layer->backward()` through the base pointer. C++ automatically picks the right version at runtime. `WeightInitializer` and `ModelIO` use `dynamic_cast<DenseLayer*>` when they need to access dense-layer-specific data.
- **Operator Overloading:** `Matrix` overloads five operators: `+` and `-` work element-wise; `*` does full matrix multiplication; `==` compares values within a tolerance of 10^{-9} ; and `<<` prints the matrix to any output stream. All operators throw `std::runtime_error` if the dimensions do not match.
- **Smart Pointers:** `std::unique_ptr<Layer>` in `NeuralNetwork::layers` frees memory automatically when the network is destroyed. There are no manual `delete` calls anywhere in the code.

3 Implementation

3.1 Development Environment

- **Language:** C++17
- **Compiler:** g++ (MinGW-w64) with `-std=c++17 -Wall`
- **IDE:** Visual Studio Code
- **Graphics library:** SFML 3.0 (Simple and Fast Multimedia Library)
- **Platform:** Windows 10/11
- **Build:** CMake (version 3.10+)

3.2 Abstract Layer Hierarchy

The `Layer` base class stores the last input it received (needed for backprop) and declares three pure-virtual functions that every subclass must implement. `DenseLayer` adds `weights` and `bias` matrices that get updated during training; `SigmoidLayer` has no learnable parameters.

```

1  // abstract base: can't use this directly, need DenseLayer or
   SigmoidLayer
2  class Layer {
3  public:
4      Matrix last_input; // saved during forward pass, needed for
                           backprop
5
6      Layer();
7      virtual ~Layer();
8
9      virtual Matrix forward(const Matrix& input) = 0;
10     virtual Matrix backward(const Matrix& grad, double lr) = 0;
11     virtual std::string get_type() = 0;
12 };
13
14 // fully connected layer: output = input * weights + bias
15 class DenseLayer : public Layer {
16 public:
17     Matrix weights; // shape: (in, out) -- rows=inputs, cols=outputs
18     Matrix bias;    // shape: (1, out)
19
20     DenseLayer(int in, int out);

```

```

21     std::string get_type() override;
22     Matrix forward(const Matrix& input) override;
23     Matrix backward(const Matrix& grad, double lr) override;
24     void set_weights(const std::vector<std::vector<double>>& w);
25 };
26
27 // applies sigmoid element-wise, no learnable parameters
28 class SigmoidLayer : public Layer {
29 public:
30     std::string get_type() override;
31     Matrix forward(const Matrix& input) override;
32     Matrix backward(const Matrix& grad, double lr) override;
33 };

```

Listing 1: Layer hierarchy — core_engine.h

3.3 Matrix Operator Overloading

Everything passing through the network—inputs, weights, outputs, gradients—is a `Matrix`. The five overloaded operators keep the math code short and readable.

```

1  // element-wise addition
2  Matrix Matrix::operator+(const Matrix& other) const {
3      if (rows != other.rows || cols != other.cols)
4          throw std::runtime_error("Matrix::operator+ size mismatch");
5      Matrix result(rows, cols);
6      for (int i = 0; i < rows; i++)
7          for (int j = 0; j < cols; j++)
8              result.data[i][j] = data[i][j] + other.data[i][j];
9      return result;
10 }
11
12 // operator* does full matrix multiply
13 Matrix Matrix::operator*(const Matrix& other) const {
14     if (cols != other.rows)
15         throw std::runtime_error("Matrix::operator* inner dimension
16             mismatch");
17     return Matrix::multiply(*this, other);
18 }
19
20 // epsilon comparison so floating-point rounding doesn't cause false
21 mismatches
22 bool Matrix::operator==(const Matrix& other) const {
23     if (rows != other.rows || cols != other.cols) return false;
24     const double epsilon = 1e-9;
25     for (int i = 0; i < rows; i++)
26         for (int j = 0; j < cols; j++)
27             if (std::fabs(data[i][j] - other.data[i][j]) > epsilon)
28                 return false;
29     return true;
30 }

```

Listing 2: Matrix operators — core_engine.cpp (selected)

3.4 Forward and Backward Propagation

The core math is shown below. For a dense layer with weight matrix $W \in \mathbb{R}^{n_{in} \times n_{out}}$ and bias $b \in \mathbb{R}^{1 \times n_{out}}$:

$$y = xW + b$$

The sigmoid activation is applied element-wise after each dense layer:

$$\sigma(x) = \frac{1}{1 + e^{-x}}, \quad \frac{d\sigma}{dx} = \sigma(x)(1 - \sigma(x))$$

Training minimises mean-squared error (MSE) loss:

$$\mathcal{L} = \frac{1}{n_{out}} \sum_{j=1}^{n_{out}} (\hat{y}_j - y_j)^2, \quad \frac{\partial \mathcal{L}}{\partial \hat{y}_j} = 2(\hat{y}_j - y_j)$$

The starting gradient is computed in `NeuralNetwork::train()` and passed back through all layers in reverse order.

```

1  // polymorphic forward: correct layer->forward() selected by vtable
   at runtime
2  Matrix NeuralNetwork::predict(Matrix input) {
3      for (auto& layer : layers)
4          input = layer->forward(input);
5      return input;
6  }
7
8  void NeuralNetwork::train(const Matrix& input,
9                           const Matrix& target, double lr) {
10     Matrix output = predict(input);
11
12     // initial gradient: 2*(prediction - target) from MSE derivative
13     Matrix grad = output - target;
14     for (int j = 0; j < grad.cols; j++)
15         grad.data[0][j] *= 2.0;
16
17     // backprop through layers in reverse
18     for (int i = static_cast<int>(layers.size()) - 1; i >= 0; i--)
19         grad = layers[i]->backward(grad, lr);
20 }
21
22 // DenseLayer backward: compute weight gradients and SGD update
23 Matrix DenseLayer::backward(const Matrix& grad, double lr) {
24     Matrix input_T      = last_input.transpose();
25     Matrix weights_grad = Matrix::multiply(input_T, grad);
26     Matrix weights_T     = weights.transpose();
27     Matrix input_grad   = Matrix::multiply(grad, weights_T);
28
29     // gradient descent update
30     for (int i = 0; i < weights.rows; i++)
31         for (int j = 0; j < weights.cols; j++)
32             weights.data[i][j] -= lr * weights_grad.data[i][j];
33     for (int j = 0; j < bias.cols; j++)
34         bias.data[0][j] -= lr * grad.data[0][j];
35
36     return input_grad;
37 }
38
39 // sigmoid derivative: s * (1 - s)

```

```

40 Matrix SigmoidLayer::backward(const Matrix& grad, double lr) {
41     Matrix result(grad.rows, grad.cols);
42     for (int i = 0; i < grad.rows; i++)
43         for (int j = 0; j < grad.cols; j++) {
44             double s = 1.0 / (1.0 + exp(-last_input.data[i][j]));
45             result.data[i][j] = grad.data[i][j] * (s * (1.0 - s));
46         }
47     return result;
48 }

```

Listing 3: Forward pass and backpropagation — core_engine.cpp

3.5 Xavier Weight Initialisation

Starting with large random weights pushes sigmoid inputs to the extremes where the gradient is nearly zero and learning stalls. Xavier initialization keeps the weight range proportional to the layer size:

$$W_{ij} \sim \mathcal{U}\left(-\sqrt{\frac{6}{n_{in} + n_{out}}}, +\sqrt{\frac{6}{n_{in} + n_{out}}}\right)$$

For the first dense layer ($n_{in} = 16$, $n_{out} = 16$) this gives a limit of $\sqrt{6/32} \approx 0.433$.

```

1  // Xavier: limit = sqrt(6 / (fan_in + fan_out))
2  void WeightInitializer::xavier(DenseLayer& layer, unsigned int seed)
3  {
4      if (layer.weights.rows <= 0 || layer.weights.cols <= 0)
5          throw std::invalid_argument(
6              "DenseLayer weight size must be positive for Xavier init"
7          );
8      double limit = std::sqrt(6.0 /
9          (layer.weights.rows + layer.weights.cols));
10     uniform(layer, -limit, limit, seed);
11 }
12 // apply Xavier to every DenseLayer in the network (skips
13 // SigmoidLayers)
14 void WeightInitializer::xavier_network(NeuralNetwork& network,
15     unsigned int seed) {
16     const auto& layers = network.get_layers();
17     int dense_index = 0;
18     for (const auto& layer : layers) {
19         if (layer->get_type() != "Dense") continue;
20         DenseLayer* dense = dynamic_cast<DenseLayer*>(layer.get());
21         xavier(*dense, seed + dense_index);
22         dense_index++;
23     }
24 }

```

Listing 4: Xavier initialisation — pipeline.cpp

3.6 Top-K Epoch Tracking with std::priority_queue

Both TrainingPipeline and SimulationUI track the three lowest-loss epochs using a max-heap. The worst of the three sits at the top; when a better epoch comes in, one pop() removes it and the heap stays at exactly $K = 3$ entries.

```

1  // max-heap: worst of the top-3 sits at the top, pop it when we go
2  // over 3

```

```

2 using EpochEntry = std::pair<double, int>;
3 std::priority_queue<EpochEntry> best_epochs;
4 const int K = 3;
5
6 for (int epoch = 0; epoch < epochs; epoch++) {
7     // ... compute average_loss for this epoch ...
8
9     best_epochs.push(EpochEntry(average_loss, epoch + 1));
10    if (static_cast<int>(best_epochs.size()) > K)
11        best_epochs.pop(); // drop the worst of the current best-3
12 }
13
14 // drain in descending order (worst-to-best), then reverse for
display
15 std::vector<EpochEntry> ranked;
16 while (!best_epochs.empty()) {
17     ranked.push_back(best_epochs.top());
18     best_epochs.pop();
19 }
20 // ranked[0] is the worst of the top-3; iterate in reverse to show
best first

```

Listing 5: Max-heap top-3 tracking — pipeline.cpp

3.7 Network Construction and Entry Point

The network is built in `main.cpp`, Xavier-initialized with seed 42 so results are consistent, then passed by reference to `SimulationUI`.

```

1 // load heart disease dataset: 16 features, 1 binary label (0=healthy
, 1=disease)
2 TrainingData data;
3 data.load_csv("heart_disease_normalized.csv", 16, 1, true);
4 std::cout << "Loaded samples: " << data.size() << std::endl;
5
6 // build the network: 16 -> 16 -> 8 -> 1
7 NeuralNetwork network;
8 network.add_layer(std::unique_ptr<Layer>(new DenseLayer(16, 16)));
9 network.add_layer(std::unique_ptr<Layer>(new SigmoidLayer()));
10 network.add_layer(std::unique_ptr<Layer>(new DenseLayer(16, 8)));
11 network.add_layer(std::unique_ptr<Layer>(new SigmoidLayer()));
12 network.add_layer(std::unique_ptr<Layer>(new DenseLayer(8, 1)));
13 network.add_layer(std::unique_ptr<Layer>(new SigmoidLayer()));
14
15 WeightInitializer::xavier_network(network, 42);
16 SimulationUI ui(network, data);
17 ui.run();

```

Listing 6: Network construction — main.cpp

4 Testing

4.1 Test Cases

Table 2: Test cases and results

TC #	Input / Action	Expected Output	Actual Output	Pass?
TC-01	load_csv() on heart_disease.csv with has_header=true	950 TrainingSample objects loaded, no exception	As expected	✓
TC-02	Matrix(2,3) + Matrix(2,3)	Correct element-wise sum matrix	As expected	✓
TC-03	Matrix(2,3) + Matrix(3,2)	runtime_error thrown	Exception caught	✓
TC-04	predict() on a sample after Xavier init (seed 42)	Output value in (0,1) (sigmoid bound)	As expected	✓
TC-05	Train 50 epochs; compare loss epoch 1 vs epoch 50	Loss at epoch 50 < loss at epoch 1	Loss decreases	✓
TC-06	save_model() then load_model(); run predict()	Predictions identical before and after save	As expected	✓
TC-07	Press S in the SFML window during training	network_save.dat written to disk	File created	✓
TC-08	CSV row with wrong column count	runtime_error with descriptive message	Exception thrown	✓

4.2 Edge Cases

- **Wrong matrix dimensions:** operator+, operator-, and operator* throw std::runtime_error with a clear message when sizes do not match.
- **Bad or missing CSV:** load_csv() throws if the file cannot be opened, if any row has the wrong number of columns, or if the file has no data rows at all.
- **Non-numeric cell in CSV:** parse_csv_row() uses std::stod and checks that every character was parsed; a letter or symbol in any cell throws an error with the row number.
- **Zero-size layer passed to Xavier:** WeightInitializer::xavier() throws std::invalid_argument if the layer has zero rows or columns.
- **Model file does not match the network:** ModelIO::load_model() checks the file header (PERSON_B_MODEL_V1), the number of dense layers, and each matrix size before loading anything. Any mismatch throws an error.

5 Results and Discussion

5.1 Results

The network trains on 950 patient records with Xavier initialization (seed 42) and a default learning rate of 0.01. The MSE loss drops steadily over epochs. The dashboard assigns a letter grade based on the current loss using the thresholds in Table 3.

Table 3: Dashboard letter-grade thresholds (from `drawDashboard()` in `interface.cpp`)

Grade	MSE Loss Condition	Interpretation
A+	$\mathcal{L} < 0.005$	Excellent convergence
A	$\mathcal{L} < 0.01$	Good convergence
B	$\mathcal{L} < 0.05$	Reasonable convergence
C	$\mathcal{L} \geq 0.05$	Still training
N/A	$\mathcal{L} = 0$	Training not started

The visualizer shows a wave moving through four node columns (input $\rightarrow$ 16 $\rightarrow$ 8 $\rightarrow$ output) at 0.22s per column. The output node turns **green** when the prediction is correct and **orange** when it is wrong. Each cycle picks a random patient, runs a forward pass, and updates the sidebar with the predicted risk, AI label, actual label, and verdict.

The 60-epoch loss graph (`std::deque<float> lossHistory`) shows recent training progress, and the top-3 best epochs panel updates as training continues.

Keyboard controls: **Space** toggles training, **Up/Down** adjusts the learning rate by 0.01, and **S** saves the current weights to `network_save.dat`.

5.2 Analysis

- Xavier initialization stops the sigmoid inputs from immediately going to the extremes, so the gradient can reach all three dense layers from the start.
- The `std::deque<float>` (capacity 60) keeps only the last 60 loss values. When it gets too full, the oldest value is dropped from the front.
- The `bestEpochs` heap in `SimulationUI` works independently from the one in `TrainingPipeline`: the UI heap tracks the session's best epochs, while the pipeline heap is only used for multi-epoch batch runs.
- The easing function `smoothstep01`: $f(t) = t^2(3 - 2t)$ makes the column reveal animation fade in smoothly instead of snapping.

6 Challenges and Solutions

6.1 Challenge 1: Storing Mixed Layer Types

Problem: Keeping `DenseLayer` and `SigmoidLayer` objects in one container requires base-class pointers, which means manual `new/delete` and the risk of memory leaks.

Solution: `NeuralNetwork::layers` is a `std::vector<std::unique_ptr<Layer>>`. Each layer is wrapped in a `unique_ptr` immediately; the vector frees everything automatically when the network is destroyed.

Takeaway: Smart pointers remove a whole category of memory bugs with no performance cost.

6.2 Challenge 2: Animation Speed Tied to Frame Rate

Problem: If the animation moves one step per frame, it runs at different speeds depending on the machine and system load.

Solution: `sf::Clock::restart()` returns the real elapsed time each frame (`float dt`). An `animElapsed` counter advances in seconds, and `SECONDS_PER_PHASE = 0.22f` sets the reveal time per column in real time, not frame count. One constant controls all timing.

Takeaway: Time-based animation is independent of frame rate and is the standard approach in game and UI development.

7 Conclusion

This project shows that a fully working neural network—with forward pass, backprop, Xavier initialization, model saving, and live visualization—can be built in plain C++17 using only OOP principles.

The abstract `Layer` class and its subclasses (`DenseLayer`, `SigmoidLayer`) show how a pure-virtual interface makes the design easy to extend: adding a new layer type only requires a new derived class, nothing else changes. The `Matrix` class shows how operator overloading turns long index loops into clean math expressions. Smart pointers in the layer stack handle memory automatically with no manual `delete` calls.

The pipeline classes—`TrainingData`, `TrainingPipeline`, `WeightInitializer`, and `ModelIO`—each hide their internals and expose only what other classes need. `SimulationUI` ties everything together in a live SFML window that makes gradient descent visible and interactive.

Every class in this project covers at least one concept from CSE142 Object Oriented Programming Techniques, making the AI Neural Network Simulator a complete, real-world application of OOP.

References

- [1] B. Stroustrup, *The C++ Programming Language*, 4th ed. Addison-Wesley, 2013.
- [2] S. Meyers, *Effective Modern C++*. O'Reilly Media, 2014.
- [3] X. Glorot and Y. Bengio, “Understanding the difficulty of training deep feedforward neural networks,” in *Proc. 13th Int. Conf. Artificial Intelligence and Statistics (AISTATS)*, 2010, pp. 249–256.
- [4] D. Jankel *et al.*, *UCI Heart Disease Dataset (Cleveland)*, UCI Machine Learning Repository, 1988. [Online]. Available: <https://archive.ics.uci.edu/dataset/45/heart+disease>.
- [5] SFML Development Team, *SFML 3.0 Documentation*. [Online]. Available: <https://www.sfm1-dev.org/documentation/3.0/>. [Accessed: May 10, 2026].
- [6] *cppreference.com — C++ reference*. [Online]. Available: <https://en.cppreference.com>. [Accessed: May 10, 2026].

A Complete Source Code

A.1 main.cpp

```

1  #include "core_engine.h"
2  #include "pipeline.h"
3  #include "interface.h"
4  #include <cstdlib>
5  #include <ctime>
6  #include <iostream>
7  #include <memory>
8
9  int main() {
10     try {
11         std::srand(static_cast<unsigned int>(std::time(nullptr)));
12
13         // load heart disease dataset: 16 features, 1 binary label (0=healthy
14         // , 1=disease)
15         TrainingData data;
16         data.load_csv("heart_disease_normalized.csv", 16, 1, true);
17         std::cout << "Loaded samples: " << data.size() << std::endl;
18
19         // build the network: 16 -> 16 -> 8 -> 1
20         NeuralNetwork network;
21         network.add_layer(std::unique_ptr<Layer>(new DenseLayer(16, 16)));
22         network.add_layer(std::unique_ptr<Layer>(new SigmoidLayer()));
23         network.add_layer(std::unique_ptr<Layer>(new DenseLayer(16, 8)));
24         network.add_layer(std::unique_ptr<Layer>(new SigmoidLayer()));
25         network.add_layer(std::unique_ptr<Layer>(new DenseLayer(8, 1)));
26         network.add_layer(std::unique_ptr<Layer>(new SigmoidLayer()));
27
28         WeightInitializer::xavier_network(network, 42);
29         std::cout << "Network ready. Launching UI..." << std::endl;
30
31         SimulationUI ui(network, data);
32         ui.run();
33
34         std::cout << "\nSimulation closed.\n";
35     } catch (const std::exception& error) {
36         std::cerr << "Fatal error: " << error.what() << "\n";
37         return 1;
38     }
39
40     return 0;
41 }

```

Listing 7: main.cpp — entry point

A.2 core_engine.h

```

1  #ifndef CORE_ENGINE_H
2  #define CORE_ENGINE_H
3
4  #include <iosfwd>
5  #include <memory>
6  #include <string>
7  #include <vector>
8
9  // 2D matrix -- everything passing through the network is a Matrix
10 class Matrix {
11 public:

```



```

12     int rows;
13     int cols;
14     std::vector<std::vector<double>> data;
15
16     Matrix(int r, int c);
17
18     void randomize();
19     Matrix transpose() const;
20
21     static Matrix multiply(const Matrix& a, const Matrix& b);
22
23     // operator* does full matrix multiply, +/- are element-wise
24     Matrix operator+(const Matrix& other) const;
25     Matrix operator-(const Matrix& other) const;
26     Matrix operator*(const Matrix& other) const;
27     bool operator==(const Matrix& other) const;
28     friend std::ostream& operator<<(std::ostream& os, const Matrix& m);
29 };
30
31 // abstract base -- can't use this directly, need DenseLayer or SigmoidLayer
32 class Layer {
33 public:
34     Matrix last_input;
35
36     Layer();
37     virtual ~Layer();
38
39     virtual Matrix forward(const Matrix& input) = 0;
40     virtual Matrix backward(const Matrix& grad, double lr) = 0;
41     virtual std::string get_type() = 0;
42 };
43
44 // fully connected layer: output = input * weights + bias
45 class DenseLayer : public Layer {
46 public:
47     Matrix weights;
48     Matrix bias;
49
50     DenseLayer(int in, int out);
51
52     std::string get_type() override;
53     Matrix forward(const Matrix& input) override;
54     Matrix backward(const Matrix& grad, double lr) override;
55     void set_weights(const std::vector<std::vector<double>>& new_weights);
56 };
57
58 // applies sigmoid element-wise, no learnable parameters
59 class SigmoidLayer : public Layer {
60 public:
61     std::string get_type() override;
62     Matrix forward(const Matrix& input) override;
63     Matrix backward(const Matrix& grad, double lr) override;
64 };
65
66 // holds the layer stack, runs forward and backward passes
67 class NeuralNetwork {
68 private:
69     std::vector<std::unique_ptr<Layer>> layers;
70
71 public:
72     void add_layer(std::unique_ptr<Layer> layer);
73     const std::vector<std::unique_ptr<Layer>>& get_layers() const;
74     Matrix predict(Matrix input);

```

```

75     void train(const Matrix& input, const Matrix& target, double lr);
76 };
77
78 #endif

```

Listing 8: core_engine.h — Matrix and Layer hierarchy

A.3 core_engine.cpp

```

1  #include "core_engine.h"
2
3  #include <cmath>
4  #include <cstdlib>
5  #include <ostream>
6  #include <stdexcept>
7  #include <utility>
8
9  Matrix::Matrix(int r, int c)
10      : rows(r), cols(c), data(r, std::vector<double>(c, 0.0)) {}
11
12 void Matrix::randomize() {
13     for (int i = 0; i < rows; i++)
14         for (int j = 0; j < cols; j++)
15             data[i][j] = ((double)rand() / RAND_MAX) * 2 - 1;
16 }
17
18 Matrix Matrix::multiply(const Matrix& a, const Matrix& b) {
19     Matrix result(a.rows, b.cols);
20     for (int i = 0; i < a.rows; i++)
21         for (int j = 0; j < b.cols; j++)
22             for (int k = 0; k < a.cols; k++)
23                 result.data[i][j] += a.data[i][k] * b.data[k][j];
24     return result;
25 }
26
27 Matrix Matrix::transpose() const {
28     Matrix result(cols, rows);
29     for (int i = 0; i < rows; i++)
30         for (int j = 0; j < cols; j++)
31             result.data[j][i] = data[i][j];
32     return result;
33 }
34
35 Matrix Matrix::operator+(const Matrix& other) const {
36     if (rows != other.rows || cols != other.cols)
37         throw std::runtime_error("Matrix::operator+ size mismatch");
38     Matrix result(rows, cols);
39     for (int i = 0; i < rows; i++)
40         for (int j = 0; j < cols; j++)
41             result.data[i][j] = data[i][j] + other.data[i][j];
42     return result;
43 }
44
45 Matrix Matrix::operator-(const Matrix& other) const {
46     if (rows != other.rows || cols != other.cols)
47         throw std::runtime_error("Matrix::operator- size mismatch");
48     Matrix result(rows, cols);
49     for (int i = 0; i < rows; i++)
50         for (int j = 0; j < cols; j++)
51             result.data[i][j] = data[i][j] - other.data[i][j];
52     return result;
53 }
54

```

```

55 Matrix Matrix::operator*(const Matrix& other) const {
56     if (cols != other.rows)
57         throw std::runtime_error("Matrix::operator* inner dimension mismatch"
58                                 );
59     return Matrix::multiply(*this, other);
60 }
61 // epsilon comparison so floating point rounding doesn't cause false
62 // mismatches
63 bool Matrix::operator==(const Matrix& other) const {
64     if (rows != other.rows || cols != other.cols) return false;
65     const double epsilon = 1e-9;
66     for (int i = 0; i < rows; i++)
67         for (int j = 0; j < cols; j++)
68             if (std::fabs(data[i][j] - other.data[i][j]) > epsilon)
69                 return false;
70     return true;
71 }
72 std::ostream& operator<<(std::ostream& os, const Matrix& m) {
73     os << "Matrix(" << m.rows << "x" << m.cols << ")\n";
74     for (int i = 0; i < m.rows; i++) {
75         os << "[ ";
76         for (int j = 0; j < m.cols; j++)
77             os << m.data[i][j] << " ";
78         os << "]\n";
79     }
80     return os;
81 }
82
83 Layer::Layer() : last_input(0, 0) {}
84 Layer::~~Layer() {}
85
86 DenseLayer::DenseLayer(int in, int out) : weights(in, out), bias(1, out) {
87     weights.randomize();
88     bias.randomize();
89 }
90
91 std::string DenseLayer::get_type() { return "Dense"; }
92
93 Matrix DenseLayer::forward(const Matrix& input) {
94     last_input = input;
95     Matrix output = Matrix::multiply(input, weights);
96     // add bias to every output node
97     for (int j = 0; j < output.cols; j++)
98         output.data[0][j] += bias.data[0][j];
99     return output;
100 }
101
102 Matrix DenseLayer::backward(const Matrix& grad, double lr) {
103     Matrix input_T = last_input.transpose();
104     Matrix weights_grad = Matrix::multiply(input_T, grad);
105     Matrix weights_T = weights.transpose();
106     Matrix input_grad = Matrix::multiply(grad, weights_T);
107
108     // gradient descent update
109     for (int i = 0; i < weights.rows; i++)
110         for (int j = 0; j < weights.cols; j++)
111             weights.data[i][j] -= lr * weights_grad.data[i][j];
112
113     for (int j = 0; j < bias.cols; j++)
114         bias.data[0][j] -= lr * grad.data[0][j];
115 }

```

```

116     return input_grad;
117 }
118
119 void DenseLayer::set_weights(
120     const std::vector<std::vector<double>>& new_weights) {
121     weights.data = new_weights;
122 }
123
124 std::string SigmoidLayer::get_type() { return "Sigmoid"; }
125
126 // sigmoid: squashes any value into (0, 1)
127 Matrix SigmoidLayer::forward(const Matrix& input) {
128     last_input = input;
129     Matrix result(input.rows, input.cols);
130     for (int i = 0; i < input.rows; i++)
131         for (int j = 0; j < input.cols; j++)
132             result.data[i][j] = 1.0 / (1.0 + exp(-input.data[i][j]));
133     return result;
134 }
135
136 // sigmoid derivative: s * (1 - s)
137 Matrix SigmoidLayer::backward(const Matrix& grad, double lr) {
138     Matrix result(grad.rows, grad.cols);
139     for (int i = 0; i < grad.rows; i++)
140         for (int j = 0; j < grad.cols; j++) {
141             double s = 1.0 / (1.0 + exp(-last_input.data[i][j]));
142             result.data[i][j] = grad.data[i][j] * (s * (1.0 - s));
143         }
144     return result;
145 }
146
147 void NeuralNetwork::add_layer(std::unique_ptr<Layer> layer) {
148     layers.push_back(std::move(layer));
149 }
150
151 const std::vector<std::unique_ptr<Layer>>&
152 NeuralNetwork::get_layers() const { return layers; }
153
154 Matrix NeuralNetwork::predict(Matrix input) {
155     for (auto& layer : layers)
156         input = layer->forward(input);
157     return input;
158 }
159
160 void NeuralNetwork::train(const Matrix& input,
161     const Matrix& target, double lr) {
162     Matrix output = predict(input);
163
164     // initial gradient: 2*(prediction - target) from MSE derivative
165     Matrix grad = output - target;
166     for (int j = 0; j < grad.cols; j++)
167         grad.data[0][j] *= 2.0;
168
169     // backprop through layers in reverse
170     for (int i = static_cast<int>(layers.size()) - 1; i >= 0; i--)
171         grad = layers[i]->backward(grad, lr);
172 }

```

Listing 9: core_engine.cpp — implementations

A.4 pipeline.h

```

1 #ifndef PIPELINE_H

```

```
2 #define PIPELINE_H
3
4 #include "core_engine.h"
5
6 #include <fstream>
7 #include <string>
8 #include <vector>
9
10 // one row from the dataset: 16 inputs + 1 target label
11 struct TrainingSample {
12     Matrix input;
13     Matrix target;
14
15     TrainingSample(const Matrix& in, const Matrix& out);
16 };
17
18 // loads and stores the dataset from a CSV file
19 class TrainingData {
20 private:
21     std::vector<TrainingSample> samples;
22     int input_cols;
23     int target_cols;
24
25     static std::string trim(const std::string& value);
26     static std::vector<double> parse_csv_row(const std::string& line,
27                                             int row_number);
28
29 public:
30     TrainingData();
31
32     void load_csv(const std::string& filename,
33                 int input_count, int target_count,
34                 bool has_header = false);
35     int size() const;
36     int input_count() const;
37     int target_count() const;
38     const std::vector<TrainingSample>& get_samples() const;
39 };
40
41 // runs training epochs and tracks per-epoch loss
42 class TrainingPipeline {
43 private:
44     static double sample_loss(NeuralNetwork& network,
45                             const TrainingSample& sample);
46
47 public:
48     static std::vector<double> train(
49         NeuralNetwork& network,
50         const TrainingData& data,
51         int epochs,
52         double learning_rate
53     );
54 };
55
56 // initializes network weights -- Xavier keeps gradients from vanishing early
57 // on
58 class WeightInitializer {
59 private:
60     static DenseLayer* as_dense_layer(Layer* layer, int layer_number);
61     static void fill_uniform(Matrix& matrix, double min_value,
62                             double max_value, unsigned int seed);
63
64 public:
```

```

64     static void uniform(DenseLayer& layer, double min_value,
65                          double max_value, unsigned int seed = 1);
66     static void xavier(DenseLayer& layer, unsigned int seed = 1);
67     static void xavier_network(NeuralNetwork& network, unsigned int seed = 1)
68     ;
69 };
70 // saves and loads model weights to a file so training isn't lost on exit
71 class ModelIO {
72 private:
73     static DenseLayer* as_dense_layer(Layer* layer, int layer_number);
74     static void write_matrix(std::ofstream& file, const Matrix& matrix);
75     static void read_matrix(std::ifstream& file, Matrix& matrix,
76                             const std::string& label);
77
78 public:
79     static void save_model(const NeuralNetwork& network,
80                           const std::string& filename);
81     static void load_model(NeuralNetwork& network,
82                            const std::string& filename);
83 };
84
85 #endif

```

Listing 10: pipeline.h — training pipeline declarations

A.5 interface.h

```

1  #ifndef INTERFACE_H
2  #define INTERFACE_H
3
4  #include <SFML/Graphics.hpp>
5  #include "core_engine.h"
6  #include "pipeline.h"
7  #include <string>
8  #include <iostream>
9  #include <deque>
10 #include <queue>
11 #include <utility>
12
13 class SimulationUI {
14 private:
15     sf::RenderWindow window;
16     sf::Font font;
17     sf::Text uiText;
18
19     float currentPrediction = 0.0f;
20     float currentTarget = 0.0f;
21     int currentSampleIndex = 0;
22     double bestLoss = 999.0;
23
24     NeuralNetwork& network;
25     TrainingData& data;
26
27     bool isTraining;
28     double learningRate;
29     int currentEpoch;
30     double lastLoss;
31
32     // last 60 epoch losses -- feeds the sparkline graph
33     std::deque<float> lossHistory;
34
35     // tracks the 3 best epochs -- max-heap so worst-of-best-3 sits at top

```

```

36     using EpochEntry = std::pair<double, int>;
37     std::priority_queue<EpochEntry> bestEpochs;
38     static const int TOP_K = 3;
39
40     // animation timing -- all time-based so speed is independent of frame
41     rate
42     sf::Clock animClock;
43     float animElapsed;
44     int lastVisualizedSample;
45     static constexpr float SECONDS_PER_PHASE = 0.22f;
46
47     // cached sigmoid output per neuron -- rebuilt each step so glow
48     // reflects actual network state
49     std::vector<std::vector<float>> nodeActivations;
50     void computeActivations();
51
52     void handleInput();
53     void render();
54     void drawNetworkGraph();
55     void drawDashboard();
56     void drawSparkline(float x, float y, float width, float height);
57
58 public:
59     SimulationUI(NeuralNetwork& net, TrainingData& tData);
60     void run();
61 };
62 #endif

```

Listing 11: interface.h — SimulationUI declaration

A.6 pipeline.cpp

```

1  #include "pipeline.h"
2
3  #include <cctype>
4  #include <cmath>
5  #include <iomanip>
6  #include <iostream>
7  #include <queue>
8  #include <random>
9  #include <sstream>
10 #include <stdexcept>
11 #include <utility>
12 #include <vector>
13
14 TrainingSample::TrainingSample(const Matrix& in, const Matrix& out) : input(
15     in), target(out) {}
16
17 std::string TrainingData::trim(const std::string& value) {
18     int start = 0;
19     int end = static_cast<int>(value.size()) - 1;
20
21     while (start <= end && std::isspace(static_cast<unsigned char>(value[
22         start]))) start++;
23     while (end >= start && std::isspace(static_cast<unsigned char>(value[
24         end]))) end--;
25
26     if (start > end) return "";
27     return value.substr(start, end - start + 1);
28 }

```

```

27  std::vector<double> TrainingData::parse_csv_row(const std::string& line, int
    row_number) {
28      std::vector<double> values;
29      std::stringstream ss(line);
30      std::string cell;
31
32      while (std::getline(ss, cell, ',')) {
33          cell = trim(cell);
34          if (cell.empty())
35              throw std::runtime_error("Empty value found in CSV row " + std::
                  to_string(row_number));
36
37          try {
38              size_t used_chars = 0;
39              double number = std::stod(cell, &used_chars);
40              if (used_chars != cell.size())
41                  throw std::runtime_error("extra characters");
42              values.push_back(number);
43          } catch (...) {
44              throw std::runtime_error("Non-numeric value found in CSV row " +
                  std::to_string(row_number));
45          }
46      }
47
48      return values;
49  }
50
51  TrainingData::TrainingData() : input_cols(0), target_cols(0) {}
52
53  void TrainingData::load_csv(const std::string& filename, int input_count,
    int target_count, bool has_header) {
54      if (input_count <= 0 || target_count <= 0)
55          throw std::invalid_argument("Input and target column counts must be
    positive");
56
57      std::ifstream file(filename);
58      if (!file.is_open())
59          throw std::runtime_error("Could not open dataset file: " + filename);
60
61      samples.clear();
62      input_cols = input_count;
63      target_cols = target_count;
64
65      std::string line;
66      int row_number = 0;
67      const int expected_cols = input_cols + target_cols;
68
69      while (std::getline(file, line)) {
70          row_number++;
71          if (has_header && row_number == 1) continue;
72          if (trim(line).empty()) continue;
73
74          std::vector<double> values = parse_csv_row(line, row_number);
75          if (static_cast<int>(values.size()) != expected_cols)
76              throw std::runtime_error(
77                  "CSV row " + std::to_string(row_number) + " has " +
78                  std::to_string(values.size()) + " columns, expected " +
79                  std::to_string(expected_cols)
80              );
81
82          Matrix input(1, input_cols);
83          Matrix target(1, target_cols);
84
85

```



```

86         for (int i = 0; i < input_cols; i++)
87             input.data[0][i] = values[i];
88         for (int j = 0; j < target_cols; j++)
89             target.data[0][j] = values[input_cols + j];
90
91         samples.push_back(TrainingSample(input, target));
92     }
93
94     if (samples.empty())
95         throw std::runtime_error("Dataset is empty after loading: " +
96             filename);
97 }
98
99 int TrainingData::size() const { return static_cast<int>(samples.size()); }
100 int TrainingData::input_count() const { return input_cols; }
101 int TrainingData::target_count() const { return target_cols; }
102
103 const std::vector<TrainingSample>& TrainingData::get_samples() const { return samples; }
104
105 double TrainingPipeline::sample_loss(NeuralNetwork& network, const
106     TrainingSample& sample) {
107     Matrix prediction = network.predict(sample.input);
108
109     if (prediction.cols != sample.target.cols)
110         throw std::runtime_error("Prediction size does not match target size");
111
112     double total = 0.0;
113     for (int j = 0; j < prediction.cols; j++) {
114         double diff = prediction.data[0][j] - sample.target.data[0][j];
115         total += diff * diff;
116     }
117     return total / prediction.cols;
118 }
119
120 std::vector<double> TrainingPipeline::train(
121     NeuralNetwork& network,
122     const TrainingData& data,
123     int epochs,
124     double learning_rate
125 ) {
126     if (epochs <= 0)
127         throw std::invalid_argument("Epoch count must be positive");
128     if (learning_rate <= 0.0)
129         throw std::invalid_argument("Learning rate must be positive");
130
131     std::vector<double> loss_history;
132     const std::vector<TrainingSample>& samples = data.get_samples();
133
134     // max-heap: worst of the top-3 sits at the top, pop it when we go over 3
135     using EpochEntry = std::pair<double, int>;
136     std::priority_queue<EpochEntry> best_epochs;
137     const int K = 3;
138
139     for (int epoch = 0; epoch < epochs; epoch++) {
140         double total_loss = 0.0;
141
142         for (const TrainingSample& sample : samples) {
143             network.train(sample.input, sample.target, learning_rate);
144             total_loss += sample_loss(network, sample);
145         }
146     }

```

```

144         double average_loss = total_loss / samples.size();
145         loss_history.push_back(average_loss);
146
147         best_epochs.push(EpochEntry(average_loss, epoch + 1));
148         if (static_cast<int>(best_epochs.size()) > K)
149             best_epochs.pop();
150
151         std::cout << "Epoch " << (epoch + 1) << "/" << epochs
152             << " - loss: " << average_loss << std::endl;
153     }
154
155     if (epochs > 1) {
156         std::vector<EpochEntry> ranked;
157         while (!best_epochs.empty()) {
158             ranked.push_back(best_epochs.top());
159             best_epochs.pop();
160         }
161         std::cout << "\n== Top " << ranked.size()
162             << " Best Epochs (lowest loss) ==>" << std::endl;
163         for (int i = static_cast<int>(ranked.size()) - 1; i >= 0; i--)
164             std::cout << " Epoch " << ranked[i].second
165                 << " | loss: " << std::fixed << std::setprecision(6)
166                 << ranked[i].first << std::endl;
167     }
168
169     return loss_history;
170 }
171
172 DenseLayer* WeightInitializer::as_dense_layer(Layer* layer, int layer_number)
173 {
174     DenseLayer* dense = dynamic_cast<DenseLayer*>(layer);
175     if (dense == nullptr)
176         throw std::runtime_error("Layer " + std::to_string(layer_number)
177             + " is not a DenseLayer");
178     return dense;
179 }
180
181 void WeightInitializer::fill_uniform(Matrix& matrix, double min_value,
182                                     double max_value, unsigned int seed) {
183     std::mt19937 generator(seed);
184     std::uniform_real_distribution<double> distribution(min_value, max_value);
185     ;
186     for (int i = 0; i < matrix.rows; i++)
187         for (int j = 0; j < matrix.cols; j++)
188             matrix.data[i][j] = distribution(generator);
189 }
190
191 void WeightInitializer::uniform(DenseLayer& layer, double min_value,
192                                 double max_value, unsigned int seed) {
193     if (min_value >= max_value)
194         throw std::invalid_argument(
195             "Minimum weight value must be less than maximum weight value");
196     fill_uniform(layer.weights, min_value, max_value, seed);
197     fill_uniform(layer.bias, min_value, max_value, seed + 1);
198 }
199
200 // Xavier: limit = sqrt(6 / (fan_in + fan_out))
201 void WeightInitializer::xavier(DenseLayer& layer, unsigned int seed) {
202     if (layer.weights.rows <= 0 || layer.weights.cols <= 0)
203         throw std::invalid_argument(
204             "DenseLayer weight size must be positive for Xavier
205             initialization");

```

```
204     double limit = std::sqrt(6.0 / (layer.weights.rows + layer.weights.cols))
205     ;
206     uniform(layer, -limit, limit, seed);
207 }
208 void WeightInitializer::xavier_network(NeuralNetwork& network, unsigned int
209 seed) {
210     const std::vector<std::unique_ptr<Layer>>& layers = network.get_layers();
211     int dense_index = 0;
212     for (const auto& layer : layers) {
213         if (layer->get_type() != "Dense") continue;
214         DenseLayer* dense = as_dense_layer(layer.get(), dense_index);
215         xavier(*dense, seed + dense_index);
216         dense_index++;
217     }
218 }
219 DenseLayer* ModelIO::as_dense_layer(Layer* layer, int layer_number) {
220     DenseLayer* dense = dynamic_cast<DenseLayer*>(layer);
221     if (dense == nullptr)
222         throw std::runtime_error("Layer " + std::to_string(layer_number)
223             + " is not a DenseLayer");
224     return dense;
225 }
226
227 void ModelIO::write_matrix(std::ofstream& file, const Matrix& matrix) {
228     file << matrix.rows << " " << matrix.cols << "\n";
229     file << std::setprecision(17);
230     for (int i = 0; i < matrix.rows; i++) {
231         for (int j = 0; j < matrix.cols; j++) {
232             file << matrix.data[i][j];
233             if (j + 1 < matrix.cols) file << " ";
234         }
235         file << "\n";
236     }
237 }
238
239 void ModelIO::read_matrix(std::ifstream& file, Matrix& matrix,
240     const std::string& label) {
241     int rows = 0, cols = 0;
242     if (!(file >> rows >> cols))
243         throw std::runtime_error("Could not read " + label + " size");
244
245     if (rows != matrix.rows || cols != matrix.cols)
246         throw std::runtime_error(
247             label + " size mismatch. File has " +
248             std::to_string(rows) + "x" + std::to_string(cols) +
249             ", network expects " +
250             std::to_string(matrix.rows) + "x" + std::to_string(matrix.cols)
251         );
252
253     for (int i = 0; i < matrix.rows; i++)
254         for (int j = 0; j < matrix.cols; j++)
255             if (!(file >> matrix.data[i][j]))
256                 throw std::runtime_error("Could not read values for " + label
257                     );
258 }
259
260 void ModelIO::save_model(const NeuralNetwork& network,
261     const std::string& filename) {
262     std::ofstream file(filename);
263     if (!file.is_open())
264         throw std::runtime_error("Could not create model file: " + filename);
```

```

264
265     const std::vector<std::unique_ptr<Layer>>& layers = network.get_layers();
266     int dense_count = 0;
267     for (const auto& layer : layers)
268         if (layer->get_type() == "Dense") dense_count++;
269
270     file << "PERSON_B_MODEL_V1\n";
271     file << dense_count << "\n";
272
273     int dense_index = 0;
274     for (const auto& layer : layers) {
275         if (layer->get_type() != "Dense") continue;
276         DenseLayer* dense = as_dense_layer(layer.get(), dense_index);
277         file << "DENSE " << dense_index << "\n";
278         write_matrix(file, dense->weights);
279         write_matrix(file, dense->bias);
280         dense_index++;
281     }
282 }
283
284 void ModelIO::load_model(NeuralNetwork& network, const std::string& filename)
285 {
286     std::ifstream file(filename);
287     if (!file.is_open())
288         throw std::runtime_error("Could not open model file: " + filename);
289
290     std::string header;
291     int dense_count = 0;
292
293     if (!(file >> header) || header != "PERSON_B_MODEL_V1")
294         throw std::runtime_error("Invalid model file format");
295     if (!(file >> dense_count) || dense_count < 0)
296         throw std::runtime_error("Invalid dense layer count in model file");
297
298     const std::vector<std::unique_ptr<Layer>>& layers = network.get_layers();
299     int network_dense_count = 0;
300     for (const auto& layer : layers)
301         if (layer->get_type() == "Dense") network_dense_count++;
302
303     if (network_dense_count != dense_count)
304         throw std::runtime_error(
305             "Model dense layer count does not match the network");
306
307     int dense_index = 0;
308     for (const auto& layer : layers) {
309         if (layer->get_type() != "Dense") continue;
310
311         std::string tag;
312         int file_dense_index = -1;
313         if (!(file >> tag >> file_dense_index) || tag != "DENSE"
314             || file_dense_index != dense_index)
315             throw std::runtime_error(
316                 "Invalid dense layer entry in model file");
317
318         DenseLayer* dense = as_dense_layer(layer.get(), dense_index);
319         read_matrix(file, dense->weights,
320             "weights for Dense layer " + std::to_string(dense_index))
321             ;
322         read_matrix(file, dense->bias,
323             "bias for Dense layer " + std::to_string(dense_index))
324             ;
325         dense_index++;
326     }
327 }

```

324 }

Listing 12: pipeline.cpp — data loading, training loop, weight init, model I/O

A.7 interface.cpp

```

1  #include "interface.h"
2  #include <sstream>
3  #include <iomanip>
4  #include <cmath>
5  #include <cstdint>
6  #include <algorithm>
7
8  SimulationUI::SimulationUI(NeuralNetwork& net, TrainingData& tData)
9      : network(net), data(tData), isTraining(false), learningRate(0.01),
10        currentEpoch(0), lastLoss(0.0), uiText(font),
11        animElapsed(0.f), lastVisualizedSample(-1) {
12
13      window.create(sf::VideoMode({1200, 800}), "AI Neural Network Simulator");
14      window.setFramerateLimit(60);
15
16      if (!font.openFromFile("arial.ttf")) {
17          std::cerr << "ERROR: Could not load arial.ttf." << std::endl;
18          exit(1);
19      }
20
21      uiText.setFont(font);
22  }
23
24  void SimulationUI::run() {
25      while (window.isOpen()) {
26          handleInput();
27
28          float dt = animClock.restart().asSeconds();
29
30          if (isTraining && data.size() > 0) {
31              std::vector<double> losses =
32                  TrainingPipeline::train(network, data, 1, learningRate);
33
34              if (!losses.empty()) {
35                  lastLoss = losses.back();
36                  if (lastLoss < bestLoss) bestLoss = lastLoss;
37
38                  bestEpochs.push(EpochEntry(lastLoss, currentEpoch + 1));
39                  if (static_cast<int>(bestEpochs.size()) > TOP_K)
40                      bestEpochs.pop();
41
42                  lossHistory.push_back(static_cast<float>(lastLoss));
43                  if (lossHistory.size() > 60) lossHistory.pop_front();
44              }
45              currentEpoch++;
46          }
47
48          int numColumns = 1;
49          for (const auto& l : network.get_layers())
50              if (dynamic_cast<DenseLayer*>(l.get())) numColumns++;
51
52          const float holdAfterComplete = 0.4f;
53          const float fullDuration =
54              SECONDS_PER_PHASE * numColumns + holdAfterComplete;
55
56          auto pickNewSample = [&]() {
57              currentSampleIndex = std::rand() % data.size();

```

```

58         const TrainingSample& sample =
59             data.get_samples()[currentSampleIndex];
60         Matrix guess = network.predict(sample.input);
61         currentPrediction = static_cast<float>(guess.data[0][0]);
62         currentTarget     = static_cast<float>(sample.target.data[0][0]);
63         animElapsed = 0.f;
64         lastVisualizedSample = currentSampleIndex;
65         computeActivations();
66     };
67
68     if (lastVisualizedSample == -1 && data.size() > 0)
69         pickNewSample();
70
71     if (isTraining && data.size() > 0) {
72         animElapsed += dt;
73         if (animElapsed >= fullDuration)
74             pickNewSample();
75     }
76
77     render();
78 }
79 }
80
81 void SimulationUI::computeActivations() {
82     nodeActivations.clear();
83     if (currentSampleIndex < 0 || data.size() == 0) return;
84
85     const TrainingSample& sample = data.get_samples()[currentSampleIndex];
86     Matrix current = sample.input;
87
88     std::vector<float> col;
89     col.reserve(current.cols);
90     for (int j = 0; j < current.cols; j++)
91         col.push_back(static_cast<float>(current.data[0][j]));
92     nodeActivations.push_back(col);
93
94     const auto& layers = network.get_layers();
95     for (const auto& layer : layers) {
96         current = layer->forward(current);
97         if (layer->get_type() == "Sigmoid") {
98             col.clear();
99             col.reserve(current.cols);
100             for (int j = 0; j < current.cols; j++)
101                 col.push_back(static_cast<float>(current.data[0][j]));
102             nodeActivations.push_back(col);
103         }
104     }
105 }
106
107 void SimulationUI::handleInput() {
108     while (const std::optional<sf::Event> event = window.pollEvent()) {
109         if (event->is<sf::Event::Closed>())
110             window.close();
111
112         if (const auto* resized = event->getIf<sf::Event::Resized>()) {
113             sf::FloatRect visibleArea({0.f, 0.f},
114                                     sf::Vector2f(resized->size));
115             window.setView(sf::View(visibleArea));
116         }
117
118         if (const auto* keyPressed = event->getIf<sf::Event::KeyPressed>()) {
119             switch (keyPressed->code) {
120                 case sf::Keyboard::Key::Space:

```

```

121         if (data.size() > 0) isTraining = !isTraining;
122         break;
123     case sf::Keyboard::Key::Up:
124         learningRate += 0.01;
125         break;
126     case sf::Keyboard::Key::Down:
127         if (learningRate > 0.01) learningRate -= 0.01;
128         break;
129     case sf::Keyboard::Key::S:
130         ModelIO::save_model(network, "network_save.dat");
131         break;
132     default:
133         break;
134     }
135 }
136 }
137 }
138
139 void SimulationUI::render() {
140     window.clear(sf::Color(22, 24, 28));
141     drawNetworkGraph();
142     drawDashboard();
143     window.display();
144 }
145
146 void SimulationUI::drawSparkline(float x, float y, float w, float h) {
147     sf::RectangleShape bg(sf::Vector2f(w, h));
148     bg.setPosition(sf::Vector2f(x, y));
149     bg.setFillColor(sf::Color(15, 17, 20));
150     bg.setOutlineThickness(1.f);
151     bg.setOutlineColor(sf::Color(45, 50, 60));
152     window.draw(bg);
153
154     if (lossHistory.empty()) return;
155
156     float maxLoss = 0.0001f;
157     for (float l : lossHistory)
158         if (l > maxLoss) maxLoss = l;
159
160     std::vector<sf::Vertex> line;
161     for (size_t i = 0; i < lossHistory.size(); ++i) {
162         float px = x + (i / 60.f) * w;
163         float py = y + h - ((lossHistory[i] / maxLoss) * h);
164         sf::Vertex point;
165         point.position = sf::Vector2f(px, py);
166         point.color = sf::Color(100, 200, 255);
167         line.push_back(point);
168     }
169
170     if (line.size() > 1)
171         window.draw(line.data(), line.size(),
172             sf::PrimitiveType::LineStrip);
173 }
174
175 void SimulationUI::drawDashboard() {
176     float windowHeight = static_cast<float>(window.getSize().y);
177
178     sf::RectangleShape sidebar(sf::Vector2f(360.f, windowHeight));
179     sidebar.setFillColor(sf::Color(22, 24, 28));
180     window.draw(sidebar);
181
182     uiText.setCharacterSize(22);
183     uiText.setFillColor(sf::Color::White);

```

```

184     uiText.setString("SYSTEM DASHBOARD");
185     uiText.setPosition({30.f, 40.f});
186     window.draw(uiText);
187
188     sf::CircleShape statusLight(6.f);
189     statusLight.setPosition({30.f, 135.f});
190     if (isTraining) {
191         float pulse = (std::sin(currentEpoch * 0.1f) * 63.f) + 192.f;
192         statusLight.setFillColor(
193             sf::Color(100, 255, 150,
194                 static_cast<std::uint8_t>(pulse)));
195     } else {
196         statusLight.setFillColor(sf::Color(255, 80, 80));
197     }
198     window.draw(statusLight);
199
200     uiText.setCharacterSize(16);
201     uiText.setFillColor(sf::Color(150, 160, 175));
202     std::stringstream controls;
203     controls << "STATUS:      " << (isTraining ? "ACTIVE" : "PAUSED") << "\n"
204             << "RATE:          " << std::fixed << std::setprecision(4)
205             << learningRate << "\n"
206             << "TOGGLE:       [ SPACE ]\n"
207             << "TUNE:         [ UP / DOWN ]\n"
208             << "EXPORT:       [ S ]";
209     uiText.setString(controls.str());
210     uiText.setPosition({55.f, 130.f});
211     window.draw(uiText);
212
213     int predLabel = (currentPrediction > 0.5f) ? 1 : 0;
214     int actLabel  = (currentTarget    > 0.5f) ? 1 : 0;
215     bool match    = (predLabel == actLabel);
216
217     uiText.setFillColor(match ? sf::Color(120, 230, 140)
218                             : sf::Color(100, 200, 255));
219     std::stringstream inference;
220     inference << "[ LIVE INFERENCE ]\n\n"
221             << "Patient ID: #" << currentSampleIndex << "\n"
222             << "AI Risk:      " << std::fixed << std::setprecision(1)
223             << currentPrediction * 100.0f << "%\n"
224             << "AI Says:      " << (predLabel == 1 ? "DISEASE" : "HEALTHY")
225             << "\n"
226             << "Actual:       " << (actLabel == 1 ? "DISEASE" : "HEALTHY")
227             << "\n"
228             << "VERDICT:      " << (match ? "MATCH" : "MISS");
229     uiText.setString(inference.str());
230     uiText.setPosition({30.f, 300.f});
231     window.draw(uiText);
232
233     uiText.setFillColor(sf::Color(150, 160, 175));
234     std::string grade = "C";
235     if (lastLoss < 0.005) grade = "A+";
236     else if (lastLoss < 0.01) grade = "A";
237     else if (lastLoss < 0.05) grade = "B";
238     else if (lastLoss == 0.0) grade = "N/A";
239
240     std::stringstream metrics;
241     metrics << "[ PERFORMANCE ]\n\n"
242             << "EPOCH:        " << currentEpoch << "\n"
243             << "CUR LOSS:     " << std::fixed << std::setprecision(6)
244             << lastLoss << "\n"
245             << "MODEL GRADE:  " << grade << "\n"
246             << "SAMPLES:      " << data.size() << "\n\n";

```



```

247
248     std::priority_queue<EpochEntry> pqCopy = bestEpochs;
249     std::vector<EpochEntry> ranked;
250     while (!pqCopy.empty()) {
251         ranked.push_back(pqCopy.top());
252         pqCopy.pop();
253     }
254     metrics << "[ TOP " << TOP_K << " BEST EPOCHS ]\n";
255     if (ranked.empty()) {
256         metrics << " (waiting for training...)";
257     } else {
258         for (int i = static_cast<int>(ranked.size()) - 1; i >= 0; i--)
259             metrics << " #" << ranked[i].second
260                 << " loss " << std::fixed << std::setprecision(5)
261                 << ranked[i].first << "\n";
262     }
263     uiText.setString(metrics.str());
264     uiText.setPosition({30.f, 460.f});
265     window.draw(uiText);
266
267     uiText.setCharacterSize(12);
268     uiText.setString("LOSS HISTORY (LAST 60 EPOCHS)");
269     uiText.setPosition({30.f, 690.f});
270     window.draw(uiText);
271
272     drawSparkline(30.f, 710.f, 300.f, 80.f);
273 }
274
275 static inline float smoothstep01(float t) {
276     if (t < 0.f) t = 0.f;
277     if (t > 1.f) t = 1.f;
278     return t * t * (3.f - 2.f * t);
279 }
280
281 static inline sf::Color lerpColor(const sf::Color& a, const sf::Color& b,
282                                   float t) {
283     if (t < 0.f) t = 0.f;
284     if (t > 1.f) t = 1.f;
285     return sf::Color(
286         static_cast<std::uint8_t>(a.r + (b.r - a.r) * t),
287         static_cast<std::uint8_t>(a.g + (b.g - a.g) * t),
288         static_cast<std::uint8_t>(a.b + (b.b - a.b) * t),
289         static_cast<std::uint8_t>(a.a + (b.a - a.a) * t)
290     );
291 }
292
293 void SimulationUI::drawNetworkGraph() {
294     const auto& all_layers = network.get_layers();
295     std::vector<DenseLayer*> denseLayers;
296     for (const auto& l : all_layers)
297         if (auto* d = dynamic_cast<DenseLayer*>(l.get()))
298             denseLayers.push_back(d);
299     if (denseLayers.empty()) return;
300
301     std::vector<std::string> inputLabels = {
302         "Age", "Sex", "Chest Pain", "Resting BP",
303         "Cholesterol", "Fasting Sugar", "Resting ECG", "Max Heart Rate",
304         "Exer. Angina", "ST Depress.", "ST Slope", "Num Vessels",
305         "Thal", "BMI", "Smoking", "Diabetes"
306     };
307     std::vector<std::string> outputLabels = { "Risk" };
308
309     int numColumns = static_cast<int>(denseLayers.size()) + 1;

```

```

310 float windowWidth      = static_cast<float>(window.getSize().x);
311 float windowHeight     = static_cast<float>(window.getSize().y);
312 float sidebarWidth     = 360.f;
313 float graphAreaWidth   = windowWidth - sidebarWidth;
314
315 float startX           = sidebarWidth + (graphAreaWidth * 0.22f);
316 float endX             = windowWidth - (graphAreaWidth * 0.15f);
317 float layerSpacingX    = (endX - startX) / (numColumns - 1);
318 float nodeRadius       = 7.f;
319
320 float fcol             = animElapsed / SECONDS_PER_PHASE;
321 int   activeColumn     = static_cast<int>(fcol);
322 float phaseProgress    = fcol - static_cast<float>(activeColumn);
323
324 auto columnReveal = [&](int col) {
325     float t = fcol - static_cast<float>(col);
326     if (t <= 0.f) return 0.f;
327     if (t >= 1.f) return 1.f;
328     return smoothstep01(t);
329 };
330
331 int   predLabel = (currentPrediction > 0.5f) ? 1 : 0;
332 int   actLabel  = (currentTarget    > 0.5f) ? 1 : 0;
333 bool  match     = (predLabel == actLabel);
334
335 const sf::Color colorIdleNode   (20, 28, 36);
336 const sf::Color colorActiveCore(60, 220, 175);
337 const sf::Color colorRimDim     (70, 100, 130, 180);
338 const sf::Color colorRimLit     (150, 240, 220, 255);
339 const sf::Color colorGlow       (80, 230, 200, 60);
340 const sf::Color colorConnIdle   (110, 120, 135, 18);
341 const sf::Color colorConnLit    (170, 235, 215, 200);
342 const sf::Color colorOutputMatch(120, 235, 150);
343 const sf::Color colorOutputMiss (255, 170, 70);
344 const sf::Color colorActual     (80, 180, 255);
345
346 for (size_t i = 0; i < denseLayers.size(); ++i) {
347     DenseLayer* d = denseLayers[i];
348     float prevX = startX + (i * layerSpacingX);
349     float currX = startX + ((i + 1) * layerSpacingX);
350
351     int inNodes  = d->weights.rows;
352     int outNodes = d->weights.cols;
353
354     float prevSpacingY = (windowHeight * 0.8f) / std::max(1, inNodes);
355     float prevStartY   =
356         (windowHeight - (prevSpacingY * (inNodes - 1))) / 2.f;
357     float currSpacingY = (windowHeight * 0.8f) / std::max(1, outNodes);
358     float currStartY   =
359         (windowHeight - (currSpacingY * (outNodes - 1))) / 2.f;
360
361     float edgeT = fcol - static_cast<float>(i);
362     float edgeBrightness = 0.f;
363     if (edgeT > 0.f && edgeT < 1.5f) {
364         edgeBrightness = std::sin(std::min(edgeT, 1.f) * 3.14159f);
365         edgeBrightness = std::max(0.f, edgeBrightness);
366     }
367     float edgeTail = (fcol > static_cast<float>(i + 1)) ? 0.25f : 0.f;
368     float edgeMix   = std::max(edgeBrightness, edgeTail);
369
370     for (int j = 0; j < outNodes; ++j) {
371         for (int k = 0; k < inNodes; ++k) {
372             double w = d->weights.data[k][j];

```

```

373         double absw = std::abs(w);
374         if (absw < 0.05) continue;
375
376         std::uint8_t baseA = static_cast<std::uint8_t>(
377             std::min(255.0, absw * 60.0 + 12.0));
378         sf::Color base = colorConnIdle;
379         base.a = baseA;
380
381         sf::Color col = lerpColor(base, colorConnLit, edgeMix);
382
383         sf::Vertex line[2];
384         line[0].position = {prevX,
385                             prevStartY + k * prevSpacingY};
386         line[1].position = {currX,
387                             currStartY + j * currSpacingY};
388         line[0].color = line[1].color = col;
389         window.draw(line, 2, sf::PrimitiveType::Lines);
390     }
391 }
392 }
393
394 for (int i = 0; i < numColumns; ++i) {
395     int nodes = (i == 0) ? denseLayers[0]->weights.rows
396                     : denseLayers[i - 1]->weights.cols;
397     float xPos      = startX + (i * layerSpacingX);
398     float spacingY  = (windowHeight * 0.8f) / std::max(1, nodes);
399     float startY    = (windowHeight - (spacingY * (nodes - 1))) / 2.f;
400
401     bool isInputCol  = (i == 0);
402     bool isOutputCol = (i == numColumns - 1);
403     bool isPulsing   = (i == activeColumn);
404
405     float reveal      = columnReveal(i);
406     float pulseEnvelope =
407         isPulsing ? std::sin(phaseProgress * 3.14159f) : 0.f;
408
409     const std::vector<float>* acts = nullptr;
410     if (i < static_cast<int>(nodeActivations.size()))
411         acts = &nodeActivations[i];
412
413     for (int n = 0; n < nodes; ++n) {
414         float yPos = startY + (n * spacingY);
415
416         float activation =
417             (acts && n < static_cast<int>(acts->size()))
418             ? (*acts)[n] : 0.5f;
419         if (activation < 0.f) activation = 0.f;
420         if (activation > 1.f) activation = 1.f;
421
422         float brightness =
423             reveal * (0.35f + 0.65f * activation)
424             + 0.35f * pulseEnvelope * activation;
425         if (brightness > 1.f) brightness = 1.f;
426
427         if (brightness > 0.05f) {
428             float haloR = nodeRadius * (2.4f + 0.6f * pulseEnvelope);
429             sf::CircleShape halo(haloR);
430             halo.setOrigin({haloR, haloR});
431             halo.setPosition({xPos, yPos});
432             sf::Color hc = colorGlow;
433             hc.a = static_cast<std::uint8_t>(
434                 std::min(255.f, hc.a * brightness * 1.6f));
435             halo.setFill_color(hc);

```

```

436         window.draw(halo);
437     }
438
439     float bodyR = nodeRadius + 0.6f * pulseEnvelope;
440     sf::CircleShape body(bodyR);
441     body.setOrigin({bodyR, bodyR});
442     body.setPosition({xPos, yPos});
443
444     sf::Color core = lerpColor(colorIdleNode, colorActiveCore,
445                               brightness);
446
447     if (isOutputCol && reveal > 0.05f) {
448         sf::Color target = match ? colorOutputMatch : colorOutputMiss
449         ;
450         core = lerpColor(colorIdleNode, target, brightness);
451     }
452
453     body.setFillColor(core);
454
455     sf::Color rim = lerpColor(colorRimDim, colorRimLit, brightness);
456     if (isOutputCol && reveal > 0.05f) {
457         rim = lerpColor(colorRimDim,
458                       match ? colorOutputMatch : colorOutputMiss,
459                       std::min(1.f, brightness + 0.3f));
460         rim.a = 255;
461     }
462     body.setOutlineThickness(1.5f);
463     body.setOutlineColor(rim);
464     window.draw(body);
465
466     if (isInputCol &&
467         n < static_cast<int>(inputLabels.size())) {
468         uiText.setString(inputLabels[n]);
469         uiText.setCharacterSize(11);
470         uiText.setFillColor(sf::Color(
471             130, 145, 160,
472             static_cast<std::uint8_t>(160 + 80 * reveal)));
473         uiText.setPosition({xPos - 115.f, yPos - 7.f});
474         window.draw(uiText);
475     }
476
477     if (isOutputCol) {
478         std::string title =
479             (nodes == 1) ? "Heart Disease" :
480             (n < static_cast<int>(outputLabels.size())
481              ? outputLabels[n] : "");
482         uiText.setString(title);
483         uiText.setCharacterSize(13);
484         uiText.setFillColor(sf::Color(255, 210, 130));
485         uiText.setPosition({xPos + 22.f, yPos - 18.f});
486         window.draw(uiText);
487
488         if (nodes == 1) {
489             uiText.setString("0 = Healthy / 1 = Disease");
490             uiText.setCharacterSize(10);
491             uiText.setFillColor(sf::Color(140, 150, 165));
492             uiText.setPosition({xPos + 22.f, yPos - 2.f});
493             window.draw(uiText);
494
495             if (reveal > 0.5f) {
496                 std::string verdictText =
497                     (predLabel == 1) ? "DISEASE" : "HEALTHY";
498                 sf::Color verdictColor =

```

```

498         match ? colorOutputMatch
499             : (predLabel == 1 ? colorOutputMiss
500                 : colorActual);
501         uiText.setString("AI: " + verdictText);
502         uiText.setCharacterSize(12);
503         uiText.setFillColor(verdictColor);
504         uiText.setPosition({xPos + 22.f, yPos + 14.f});
505         window.draw(uiText);
506
507         std::stringstream conf;
508         conf << std::fixed << std::setprecision(1)
509             << (currentPrediction * 100.0f) << "% risk";
510         uiText.setString(conf.str());
511         uiText.setCharacterSize(10);
512         uiText.setFillColor(sf::Color(180, 190, 200));
513         uiText.setPosition({xPos + 22.f, yPos + 30.f});
514         window.draw(uiText);
515     }
516 }
517
518 if (isOutputCol && reveal > 0.5f && nodes == 1) {
519     float boxSize = nodeRadius * 3.4f;
520     sf::RectangleShape highlight(
521         sf::Vector2f(boxSize, boxSize));
522     highlight.setOrigin({boxSize / 2.f, boxSize / 2.f});
523     highlight.setPosition({xPos, yPos});
524     highlight.setFillColor(sf::Color::Transparent);
525     highlight.setOutlineThickness(1.5f);
526     sf::Color frameCol =
527         match ? colorOutputMatch : sf::Color(255, 220, 90);
528     frameCol.a = static_cast<std::uint8_t>(
529         180 * std::min(1.f, (reveal - 0.5f) * 2.f));
530     highlight.setOutlineColor(frameCol);
531     window.draw(highlight);
532 }
533 }
534 }
535 }
536 }

```

Listing 13: interface.cpp — SFML window, training loop, network visualisation

B Installation and Usage

B.1 Prerequisites

- **Compiler:** A C++17-capable compiler (e.g. g++ or MSVC)
- **Build system:** CMake version 3.10 or higher
- **Graphics:** SFML 3
- **Font:** arial.ttf in the project root
- **Dataset:** heart_disease_normalized.csv in the project root

B.2 Build Instructions

Step 1 — Generate build files:

```
cmake -S . -B build
```

Listing 14: Configure with CMake

Step 2 — Compile the executable:

```
cmake --build build
```

Listing 15: Build with CMake

Step 3 — Run the simulation:

Execute the compiled `main.exe` file found inside the `build/` folder.

Controls once the window opens:

Space — toggle training on / off

Up / Down — increase / decrease learning rate by 0.01

S — save current weights to `network_save.dat`

Close window — exit